Object-Oriented Programming

Assignment #5 GUI: Qt Minesweeper

학번: 20180400

이름: 신민석

명예서약(Honor code)

"나는 이 프로그래밍 과제를 다른 사람의 부적절한 도움 없이 완수하였습니다."

목차

Object-Oriented Programming 1

1. 프로그램 개요 2
2. 프로그램 구조 및 구현 설명 2

   # 클래스 설명 2

   # Class Diagram 2

   # 메소드 구현 설명 3
3. 프로그램 실행 및 예제 시연 6

   # 컴파일 및 실행 6
4. Discussion & Improvements 8
5. References 8

1. 프로그램 개요

이번 프로그램에서 대표적인 gui 프레임워크인 qt의 기본 구성을 파악하고, 이를 응용하여 지뢰찾기 게임을 구현한다. 클래스의 상속의 실질적 유용성과 활용을 익힌다.

2. 프로그램 구조 및 구현 설명

클래스 설명

본 프로그램에서 기본적으로 구현되어 있는 QWidgets, QObject과 같은 클래스가 강력하므로, 이러한 클래스를 상속하여 원하는 기능을 구현한다. 따로 상속 제작한 클래스는 추가로 설명하도록 한다.

Class Diagram

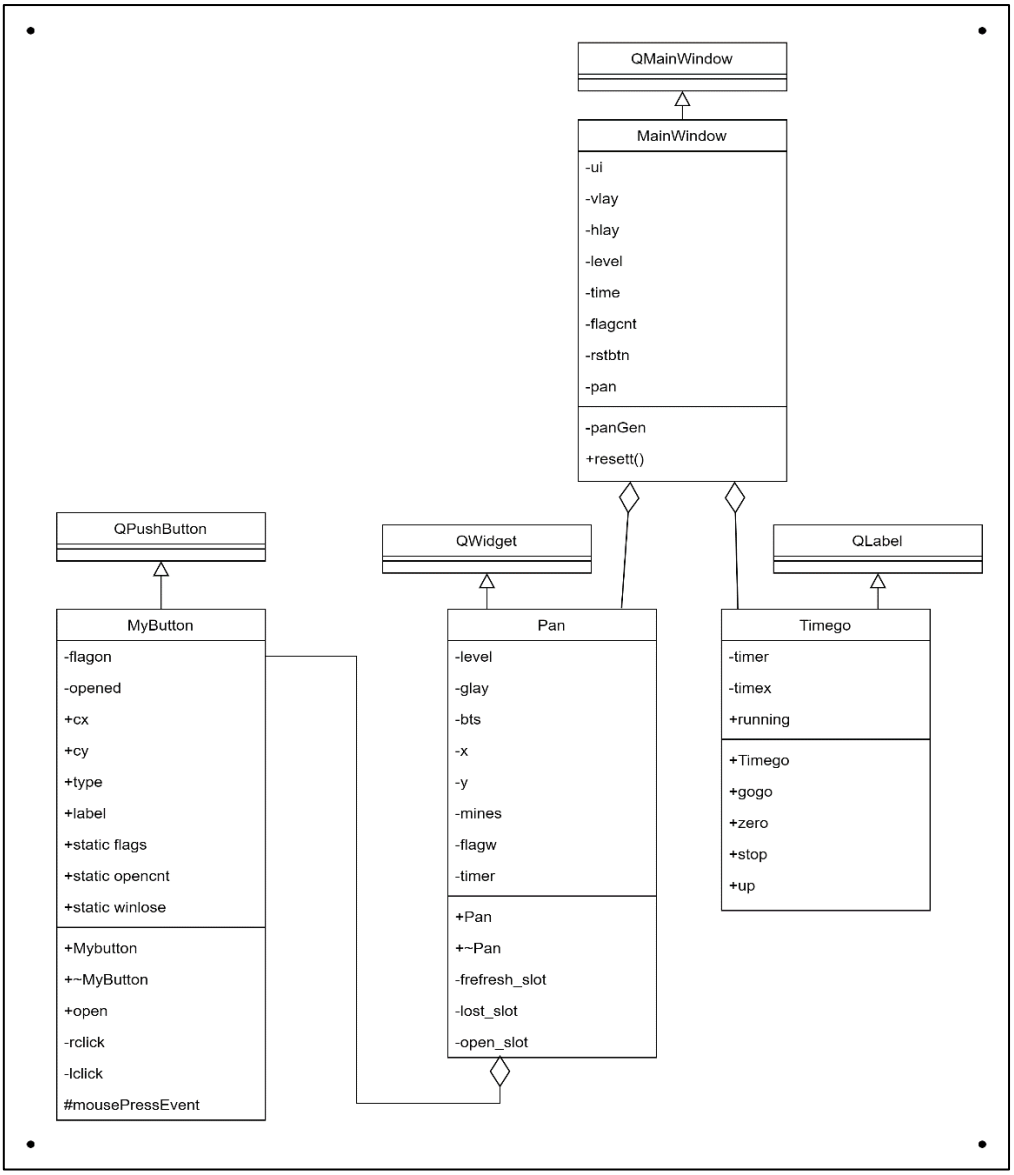


Figure 1 Class diagram

- Pan

Pan 클래스는 게임판을 의미한다. QWidget 클래스를 상속하였으며, 게임 진행의 대부분은 이 클래스에서 구현되었다. 이 클래스는 MyButton 클래스와 has-a-relationship 관계이다

- MyButton

MyButton클래스는 QPushButton 클래스를 상속하여 만든 클래스이다. 게임의 진행에 맞도록 해당 블록에서의 flag,open,type과 같은 정보들이 저장되며, 위의 Pan 클래스와 함께 작동하여 게임 진행의 핵심을 구성한다. Static 멤버 변수를 통해 각 블록이 하나의 공통 정보를 참조할 수 있다.

- timego

QLabel 클래스를 상속하여 만들어진 타이머 클래스이다. 외부로부터 신호를 받아 시작, 정지, 리셋 등의 행동을 할 수 있다. 특징은 시간을 업데이트 하는 주기는 임의적으로 설정되었다는 것으로, 시간 표출과 시간 측정이 분리되어 있는 구조이다. 이 이유로는 QTime(100)의 경우 오차가 크고 지속적으로 누적되어 별도의 QElapsedTime 객체를 생성하여 시간 측정에 사용하였기 때문이다.

- MainWindow

MainWindow는 게임이 진행되는 창이며, 다수의 클래스를 멤버로 소유한 has a relationship을 가지고 있다. 예시로 Pan, QVBoxLayout, QWidget, QLabel 등이 있다.

메소드 구현 설명

1) <Constructor> - MainWindow (mainwindow.cpp)

   MainWindow의 생성자에서는 기초적인 색상 서식을 지정하고, 게임에서 필수적으로 필요한 게임 ui 요소를 동적을 할당한다. 이는 타이머, 레벨선택기, 플레그카운터, 리셋버튼, H/V 레이아웃이 있다. 또한, Connect를 통하여 객체와 내부slot(reset())을 연결하여 준다.

2) <Destructor> - MainWindow (mainwindow.cpp)

   객체 생성시 동적 할당된 모든 요소들을 deallocate한다.

3) Resett - MainWindow (mainwindow.cpp)

   타이머를 0으로 세팅하며, 게임의 정보를 담는 static 변수들을 0으로 만든다. 그 후 panGen함수를 호출한다. Connect를 통해 레벨선택기 및 리셋버튼과 연동되었다.

4) panGen - MainWindow (mainwindow.cpp)

   panGen은 게임판을 생성하는 함수로, Pan 객체를 동적으로 새로 할당하여 화면에 띄워준다. 이 때 기존 판은 delete 된다.

5) <Constructor> - Timego (timego.h)

내부 QTimer와 내부 slot을 connect한다. 또한 상위 클래스인 QLabel 생성자를 호출한다.

6) Gogo – Timego (timego.cpp)

만약 시간이 작동중이라면 행동이 없다. 시간이 정지 상태일 경우 시간 측정을 시작한다.

7) zero – Timego (timego.cpp)

시간을 0으로 초기화한다.

8) stop – Timego (timego.cpp)

시간흐름을 정지한다.

9) up – Timego (timego.cpp)

시간을 업데이트하는 slot으로, QElapsedTimer의 결과를 QLabel에 표시한다.

10) <Constructor> - Pan (pan.cpp)

새 지뢰판을 생성한다. 지뢰판 내부 QGridLayout에 MyButton 인스턴스를 동적으로 할당하여 삽입한다. 내부 MyButton은 connect를 통해 지뢰판과 신호를 주고받는다.

판을 생성할 시에는 인자로 받은 level 값에 따라 적절한 폰트 크기 및 지뢰 및 블록 개수를 결정한다.

랜덤 지뢰생성을 위해 vector와 shuffle 알고리즘을 사용하였다. 지뢰판의 <y,x> 좌표를 std::pair로 만들어 vector에 포함시킨 후 shuffle을 진행한다. 그 후 처음 10,40,99 개 원소들을 지뢰로 세팅한다.

그 후에 지뢰가 아닌 박스들에 대해 주변 지뢰 수를 카운팅하여 MyButton.type 데이터로 삽입한다.

11) <Destructor> - Pan (pan.cpp)

지뢰판을 제거하며 내부에 할당된 메모리를 deallocate한다.

12) Frefresh_slot - Pan (pan.cpp)

사용자가 무언가 행동을 했을 때 flags 수를 refresh한다. 또한 열린 박스 수와 지뢰 수를 비교하여 우승 상태를 감지한다.
우승했을 경우 모든 박스에 대해 색을 변경하며, 이는 open()을 통한다.

13) Lost_slot - Pan (pan.cpp)

사용자가 지뢰를 눌렀을 때 반응한다. 사용자가 졌음을 출력하고 승패 상태를 -1로 세팅

한다. 이 후 모든 지뢰를 오픈한다 (open()).

14) Open_slot - Pan (pan.cpp)

빈칸인 박스(주변 지뢰 없음) 가 열릴 때 반응한다. 해당 열린 박스 주변의 8개 박스에 대하여 open 시킨다.

15) <Constructor> - MyButton (mybutton.cpp)

버튼 클릭에 대한 signal을 내부 slot에 connect 한다.

16) <Destructor> - MyButton (mybutton.cpp) = default destructor

17) mousePressEvent - MyButton (mybutton.cpp)

마우스 우 클릭에 대하여 별도의 신호를 준다.

18) rclick - MyButton (mybutton.cpp)

마우스 우클릭 시에 수행된다. 게임 진행상태이고(!winlose), 닫힌 박스(!opened) 일 경우 flag를 on/off한다. 색상과 flag 개수가 업데이트된다.

19) lclick- MyButton (mybutton.cpp)

마우스 좌클릭 시에 수행된다. 게임 진행상태이고(!winlose), 닫힌 박스(!opened)일 경우에 작동한다.
지뢰일 경우 패배 signal(emit lost())을 발생시키고, else, 해당 블록 open()을 호출한다.

20) open - MyButton (mybutton.cpp)

해당 블록을 오픈한다. 만약 승패가 우승상태일 경우, 열린 박스를 cyan 상태로 한다.

만약 빈 블록일 경우(type==0), 색을 세팅하고 오픈하며, 주변 박스들이 자동으로 열리도록 opensignal을 발생시킨다. 만약 flag 되어 있었다면 flag를 해제하고 되돌린다.

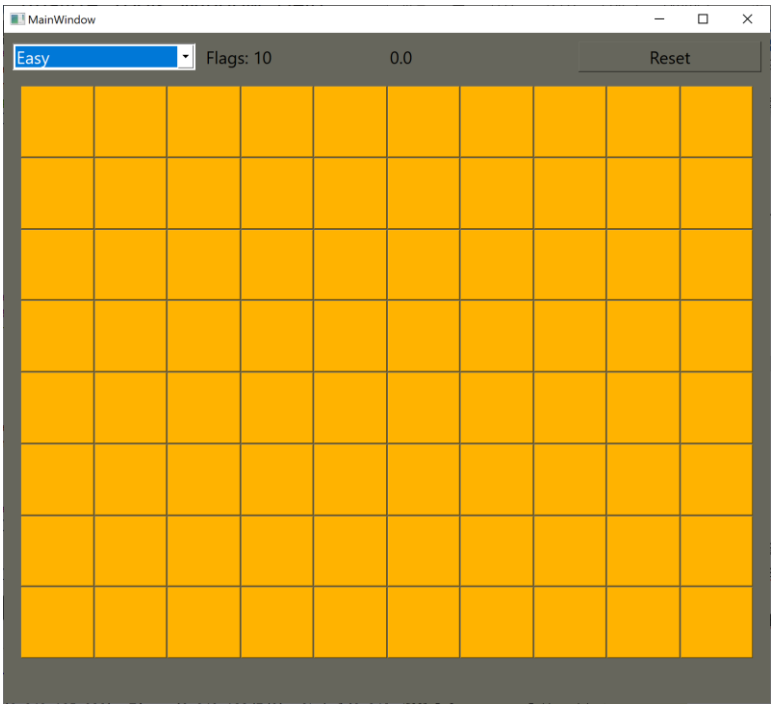
지뢰일 경우, 승패가 우승상태라면 pink 상태로 하며, 그 외에는 red 상태로 한다. 이 코드는 사용자가 지뢰를 누르면서 실행되는 것이 아니라, lost 로 승패가 결정된 후에 실행된다.

숫자일 경우, 빈 블록과 동일한 방법으로 오픈하나, opensignal을 발생시키지 않는 점이 차이점이다.

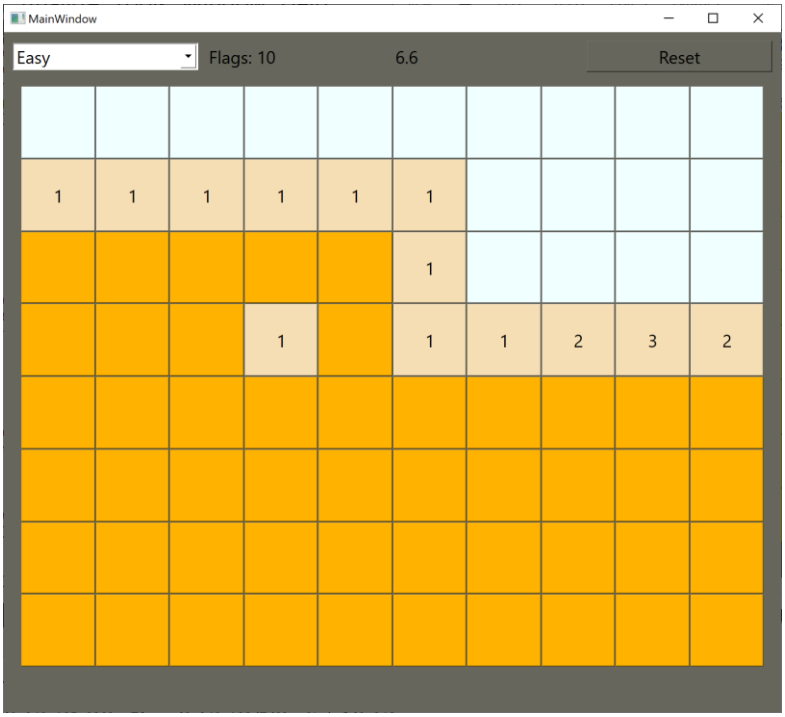
3. 프로그램 실행 및 예제 시연

본 프로그래밍 과제는 윈도우 Qt Creator 9.0.0 (Qt 6.4.1) 환경에서 진행하였다.

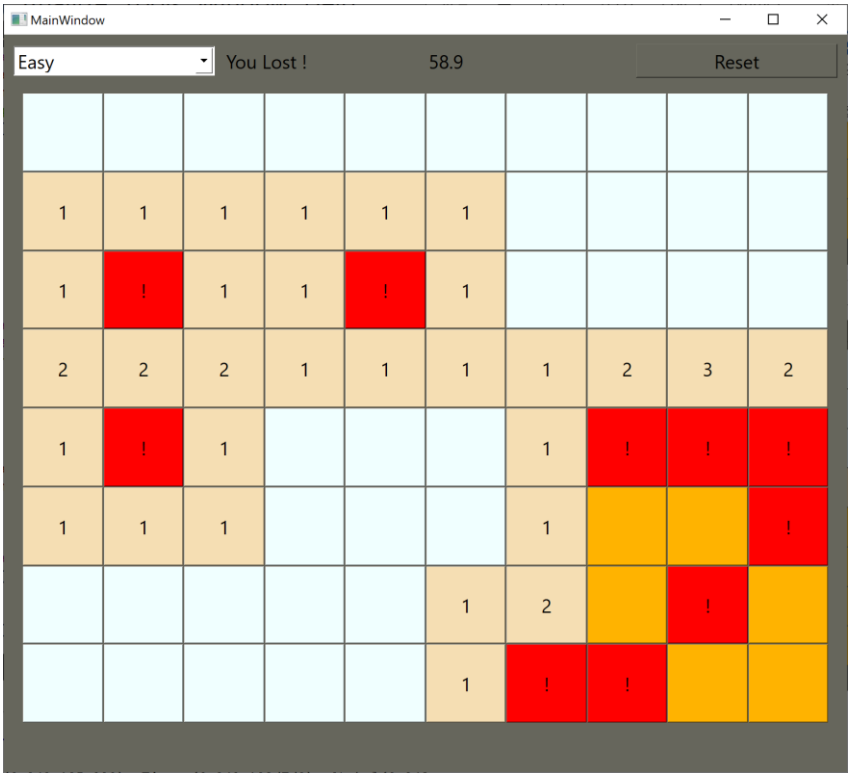
컴파일 및 실행



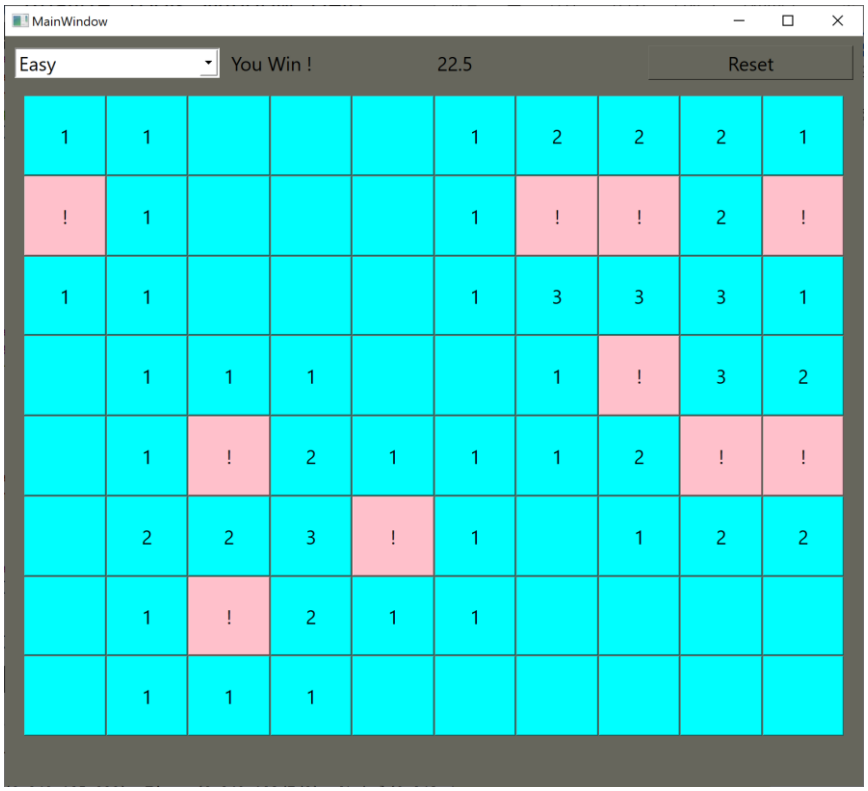
처음 실행한 결과이다. Easy로 세팅되어 있고 시간은 정지이다.



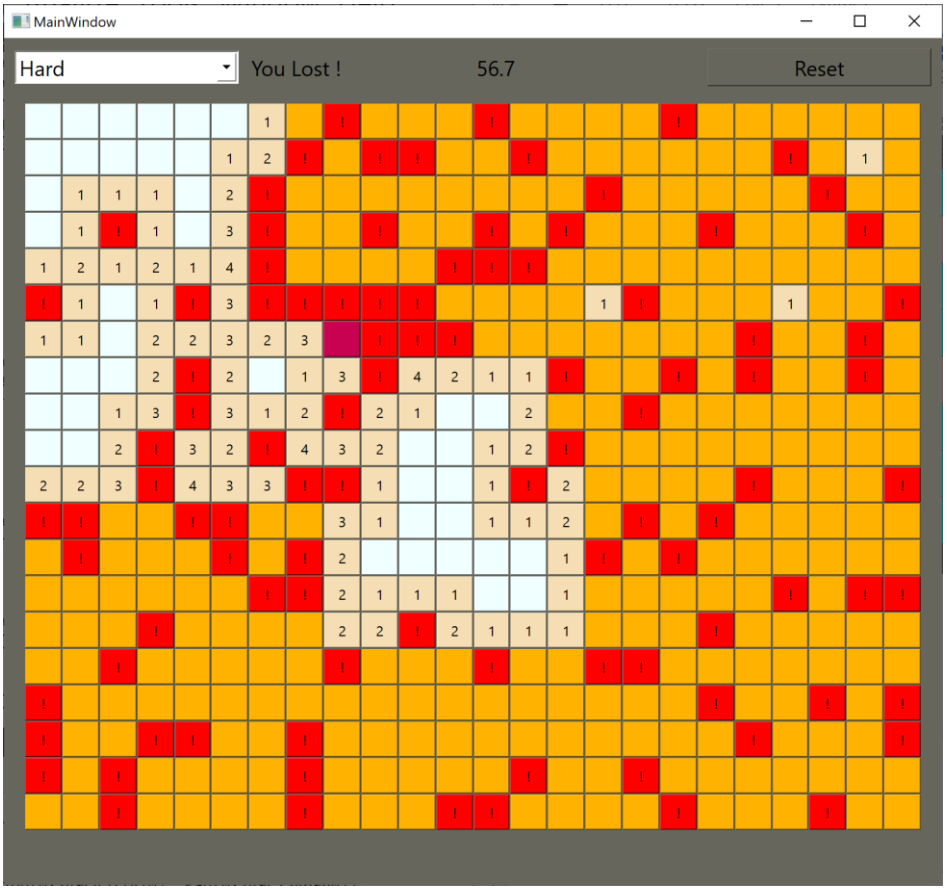
사용자가 블록을 건드리면 시간이 흐른다.



지뢰를 클릭하면 시간이 멈추고 패배하며, 모든 지뢰가 드러난다.



리셋을 누르고 우승한 결과이다.



난이도 변경이 잘 작동한다.

4. Discussion & Improvements

- Hard 모드 플레이시에 첫 클릭 때 패배하는 경우가 잦다. 윈도우 기본 지뢰찾기는 처음에 지뢰가 안 나온다고 들었는데 첫 클릭 이후에 랜덤지뢰 생성을 하면 구현할 수 있을 것으로 생각한다.

- QTimer의 경우 정확도가 매우 낮았다. QTimer의 기본 값이 Coarse 모드이므로 정밀한 시간 측정을 할 때는 파라미터를 손볼 필요가 있겠다. 이번 assignment에서는 QTimer로 시간 측정을 하지 않고 QElapsedTimer를 사용하였다.

5. References

- https://cplusplus.com/reference/algorithm/shuffle/

- [Qt] QString의 문자열 처리 (norux.me)